

2.40. La fonction `spriteTaille()` — dessiner une fois, choisir la taille

`#include <spriteTaille>` : `spriteTaille(numero, x, y, DESSIN, taille)` pose un dessin à la taille qu'on veut. Le dessin n'est écrit qu'UNE fois, à sa taille standard ; c'est le compilateur qui l'agrandit.

1. Ce qu'il faut comprendre

Dessiner une seule fois. Le rond est écrit **une seule fois**, dans `tuile_ROND.cpp`, à sa taille standard : 8×8 . Il n'y a pas de « grand rond » dans le programme. C'est dans l'appel qu'on dit à quelle taille le montrer.

La ligne à écrire : `#include <spriteTaille>`.

Ses arguments : `spriteTaille(numero, x, y, DESSIN, taille)`. `numero` est le **premier** lutin employé ; `x` et `y` le coin en haut à gauche, en pixels ; `DESSIN` le nom d'une `Tuile` ou d'un `Perso` du programme ; `taille` combien de fois plus grand : **1** sa taille, **2** deux fois, **3** trois fois...

Ici : `spriteTaille(0, 36, 60, ROND, 1)` pose le rond tel qu'il est dessiné (8×8) ; `spriteTaille(1, 96, 44, ROND, 3)` pose **le même rond** trois fois plus grand : 24×24 .

Comment le compilateur agrandit. La console ne sait pas agrandir pendant que le jeu tourne. Le compilateur redessine donc la forme **avant**, à la compilation : il fait le tour du dessin, garde les vrais coins pointus, arrondit les marches d'escalier, puis redessine à la nouvelle taille avec un contour d'un pixel. Le grand rond est **rond**, pas fait de gros pixels.

Avec combien de lutins ? La forme agrandie est coupée en carrés de 8×8 ; chaque carré qui n'est pas vide prend **un lutin**, à partir de `numero`. En taille 3, le rond fait $24 \times 24 : 3 \times 3 = 9$ lutins, les numéros 1 à 9. Le premier rond a pris le lutin 0 : les deux ne se chevauchent pas.

Pas de limite de taille. La console n'a que 40 lutins, et n'en montre que 10 sur une même ligne. Tant que la forme y tient, elle est faite de lutins : elle passe par-dessus le décor et peut bouger. **Au-delà, elle est dessinée dans le fond de l'écran**, case par case : là, plus de limite. Taille 12, 18, 40, 100... : la forme est toujours affichée. Plus grande que l'écran, on en voit la partie qui tombe dedans.

Dans le fond, la place s'écrit en clair (`spriteTaille(0, -80, -88, ROND, 40)`), par exemple, pour centrer un rond de 320 pixels) : le compilateur doit savoir quelle partie tombe dans l'écran. Elle peut être négative, c'est-à-dire commencer hors de l'écran. Le `numero` n'y sert pas.

La taille s'écrit en clair — `3`, pas une variable — : c'est le compilateur qui dessine, avant que le jeu ne tourne.

Si tu modifies le petit rond, dans `tuile_ROND.cpp`, toutes les tailles changent ensemble à la compilation suivante : il n'y a qu'un dessin.

La même logique dans les autres fonctions : la taille peut aussi s'écrire en dernier argument de `sprite(0, 36, 60, ROND, 4)`, `sprite16(0, 40, 40, HEROS, 3)`, `sprite32(0, 0, 0, BOSS, 2)`, `spriteDerriere(0, x, 72, ROND, 4)` (derrière le décor) et `poser(2, 3, ROND, 5)` (toujours dans le fond, colonne et ligne en clair). Pour les trois `sprite`, seul un nombre de **2 ou plus** est une taille : `0`, `1` et `MIROIR_X`, `DERRIERE`... gardent leur ancien sens, celui des options.

Essaie : change le `3` en `2`, puis en `4`. Puis remplace la seconde ligne par `spriteTaille(1, 8, 0, ROND, 18);` : un rond de 144 pixels, toute la hauteur de l'écran. Enfin, dessine ta propre forme dans `tuile_ROND.cpp` (une goutte, une maison) et regarde-la à toutes les tailles.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **825 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          57 o ← la console, à l'allumage – la mise en route de la console
main()             262 o ← démarrage – ta fonction
VBlank             8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage      28 o ← image() – attend l'image suivante
AttendreVBlank     16 o ← démarrage – attend que l'écran soit libre
AttendreAcces      12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte        10 o ← texte() – écrit un texte, lettre par lettre
RangerLutins       11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles       94 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte       14 o ← démarrage – vide les deux cartes de l'écran
Tuiles             272 o ← CopierTuiles – 7 lettres sur 43, et 10 tuiles dessinées
DonneesTransfert   10 o ← InstallerTransfert – la petite routine qui copie les lutins
"TAILLE 1"         8 o ← texte() – un texte gravé
"TAILLE 3"         8 o ← texte() – un texte gravé

Lettres gravées : AEILT13
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, LireManette, Notes
Fonctions du langage appelées :
  sprite()      × 10   230 o sur place, dans main()
  texte()       × 2    22 o sur place, dans main() + EcrireTexte (10 o)
  image()       × 1     6 o sur place, dans main() + AttendreImage (28 o)
  spriteTaille() × 2     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <spriteTaille> : dessiner une fois, choisir la taille ----
// UN seul dessin, ROND (8 × 8). À gauche en taille 1, à droite en taille 3 (24 × 24).

#include <texte>          // texte() : écrire un mot
#include <spriteTaille>    // spriteTaille() : un dessin à la taille qu'on veut
#include <Tuile>           // Tuile : un dessin de 8 × 8
#include "tuile_ROND.cpp" // le rond, dessiné une seule fois
```

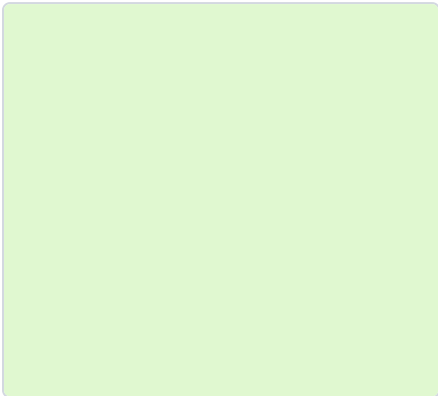
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  spriteTaille(0, 36, 60, ROND, 1); // taille 1 : 8 × 8, le lutin 0
  spriteTaille(1, 96, 44, ROND, 3); // taille 3 : 24 × 24, les lutins 1 à 9
  texte(1, 10, "TAILLE 1");         // sous celui de gauche
  texte(10, 10, "TAILLE 3");        // sous celui de droite

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Le même rond deux fois : petit à gauche (TAILLE 1), trois fois plus grand et bien rond à droite (TAILLE 3).

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ taille 1 : le rond en (36, 60), un seul lutin
- ✓ taille 3 : neuf lutins en carré de 24, à partir de (96, 44) (lutin 9 en 112, 60)
- ✓ les deux tailles sont écrites